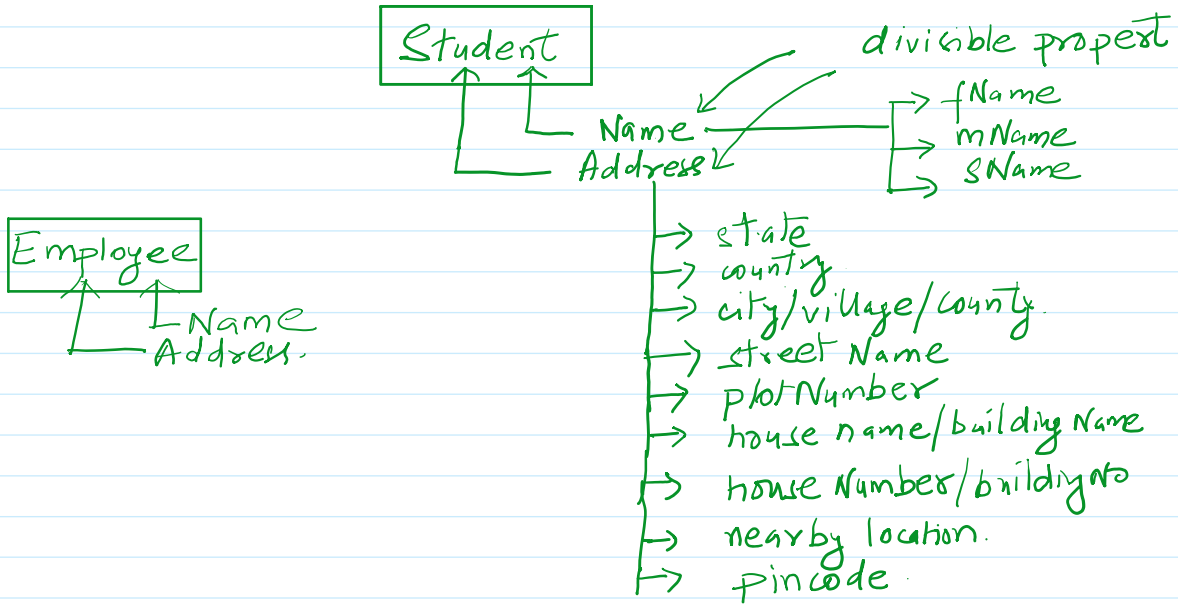
Aggregation in Java

15 December 2025 20:17

Aggregation in Java represents **"HAS-A"** relationship between two classes, where one class (the whole) contains an instance or reference to another class (the part), but the part can exist independently with its own life cycle.

Key Characteristics

Segregation is a weaker form of association compared to composition. As the contained object survives even if the container is destroyed, for instance, Books can exist outside a library. It promotes code reuse by allowing classes to share functionality without tight coupling. This relationship is typically implemented using object reference as fields in the aggregating class.

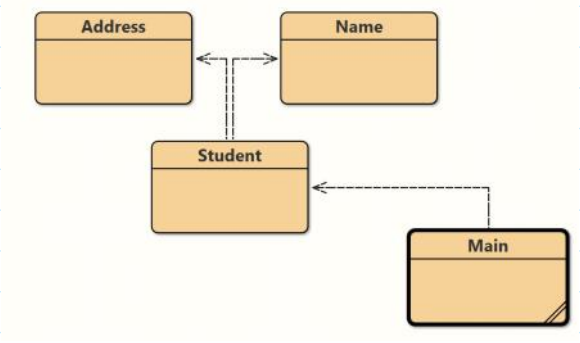


Why use Aggregation?

Ans: For Code Reusability.

When use Aggregation?

- Code reuse is also best achieved by aggregation when there is a no **is-a** relationship.
- Inheritance should be used only if the relationship **"IS-A"** is maintained throughout the lifetime of the objects involved; Otherwise, aggregation is the best choice.



package Aggregation;

```java
/**
 * Write a description of class Name here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */

public class Name {
    private String fName, mName, sName;

    public Name() {

    }

    /**
     * @param fName
     * @param mName
     * @param sName
     */
    public Name(String fName, String mName, String sName) {
        super();
        this.fName = fName;
        this.mName = mName;
        this.sName = sName;
    }

    /**
     * @return the fName
     */
    public String getfName() {
        return fName;
    }

    /**
     * @param fName the fName to set
     */
    public void setfName(String fName) {
        this.fName = fName;
    }

    /**
     * @return the mName
     */
    public String getmName() {
        return mName;
    }

    /**
     * @param mName the mName to set
     */
    public void setmName(String mName) {
        this.mName = mName;
```

```
    }

    /**
     * @return the sName
     */
    public String getsName() {
        return sName;
    }

    /**
     * @param sName the sName to set
     */
    public void setsName(String sName) {
        this.sName = sName;
    }

    @Override
    public String toString() {
        return "Name [fName=" + fName + ", mName=" + mName + ", sName=" + sName + "]";
    }
}

package Aggregation;

/**
 * Write a description of class Address here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */

public class Address {
    private String streetName, pincode, state, cityOrVillage;
    private String houseNameOrBuildingName;
    public Address(){} //default constructor
    /**
     * @param streetName
     * @param pincode
     * @param state
     * @param cityOrVillage
     * @param houseNameOrBuildingName
     */
    public Address(String streetName, String pincode, String state, String
cityOrVillage,
    String houseNameOrBuildingName) {
        super();
        this.streetName = streetName;
        this.pincode = pincode;
        this.state = state;
        this.cityOrVillage = cityOrVillage;
        this.houseNameOrBuildingName = houseNameOrBuildingName;
    }
```

```
    /**
     * @return the streetName
     */
    public String getStreetName() {
        return streetName;
    }

    /**
     * @param streetName the streetName to set
     */
    public void setStreetName(String streetName) {
        this.streetName = streetName;
    }

    /**
     * @return the pincode
     */
    public String getPincode() {
        return pincode;
    }

    /**
     * @param pincode the pincode to set
     */
    public void setPincode(String pincode) {
        this.pincode = pincode;
    }

    /**
     * @return the state
     */
    public String getState() {
        return state;
    }

    /**
     * @param state the state to set
     */
    public void setState(String state) {
        this.state = state;
    }

    /**
     * @return the cityOrVillage
     */
    public String getCityOrVillage() {
        return cityOrVillage;
    }

    /**
     * @param cityOrVillage the cityOrVillage to set
     */
    public void setCityOrVillage(String cityOrVillage) {
```

```
        this.cityOrVillage = cityOrVillage;
    }

    /**
     * @return the houseNameOrBuildingName
     */
    public String getHouseNameOrBuildingName() {
        return houseNameOrBuildingName;
    }

    /**
     * @param houseNameOrBuildingName the houseNameOrBuildingName to set
     */
    public void setHouseNameOrBuildingName(String houseNameOrBuildingName) {
        this.houseNameOrBuildingName = houseNameOrBuildingName;
    }

    @Override
    public String toString() {
        return "Address [streetName=" + streetName + ","
        + " pincode=" + pincode + ", state=" + state +
        ", cityOrVillage=" + cityOrVillage +
        ", houseNameOrBuildingName=" + houseNameOrBuildingName + "]";
    }
}

package Aggregation;
import java.util.*;

/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
    private int rollNum;
    private Name studentName;
    private Address address;

    public Student(){
        studentName = new Name();
        address = new Address();
    }

    public void inputDetails(){
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter roll number: ");
        rollNum = sc.nextInt();
        sc.nextLine(); // to remove int buffer.
        System.out.print("Enter first name: ");
        studentName.setfName(sc.nextLine());
```

```
        System.out.print("Enter middle name: ");
        studentName.setmName(sc.nextLine());
        System.out.print("Enter surname name: ");
        studentName.setsName(sc.nextLine());

        System.out.print("\nEnter the details of Address: ");
        System.out.print("\nEnter street name: ");
        address.setStreetName(sc.nextLine());
        System.out.print("Enter state: ");
        address.setState(sc.nextLine());
        System.out.print("Enter pincode: ");
        address.setPincode(sc.nextLine());
        System.out.print("Enter your city or village: ");
        address.setCityOrVillage(sc.nextLine());
        System.out.print("Enter your house name or building name: ");
        address.setHouseNameOrBuildingName(sc.nextLine());
    }

    @Override
    public String toString(){
        return "\nName: "+studentName.toString() +
        "\nAddress: \n" + address.toString();
    }
}


package Aggregation;


/**
 * Write a description of class Main here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Main
{
    public static void main(String[] args){
        System.out.print("\f");
        int numOfStudents = Integer.parseInt(args[0]);
        Student[] st = new Student[numOfStudents];
        int i;
        for(i = 0; i < numOfStudents; i++){
            //allocating the memory for the object
            st[i] = new Student();
            st[i].inputDetails(); //we insert details of the student
        }

        //printing the details of each student
        for(Student s : st){
            System.out.print("\n" + s);
        }
    }
```

```
}

Output:
   Enter roll number: 1
   Enter first name: Andy
   Enter middle name: J
   Enter surname name: Kole

   Enter the details of Address:
   Enter street name: Hudson Bay
   Enter state: California
   Enter pincode: 112003
   Enter your city or village: Red wood
   Enter your house name or building name: Halloween
   Enter roll number: 2
   Enter first name: Trisha
   Enter middle name: K
   Enter surname name: Sharma

   Enter the details of Address:
   Enter street name: Park Street
   Enter state: West Bengal
   Enter pincode: 700017
   Enter your city or village: Kolkata
   Enter your house name or building name: Mangal
   Enter roll number: 3
   Enter first name: Uma
   Enter middle name: K
   Enter surname name: Singh

   Enter the details of Address:
   Enter street name: Alipore
   Enter state: West Bengal
   Enter pincode: 700027
   Enter your city or village: Kolkata
   Enter your house name or building name: KrishnaKunj


   Name: Name [fName=Andy, mName=J, sName=Kole]
   Address:
   Address [streetName=Hudson Bay, pincode=112003, state=California, cityOrVillage=Red
   wood, houseNameOrBuildingName=Halloween]

   Name: Name [fName=Trisha, mName=K, sName=Sharma]
   Address:
   Address [streetName=Park Street, pincode=700017, state=West Bengal,
   cityOrVillage=Kolkata, houseNameOrBuildingName=Mangal]

   Name: Name [fName=Uma, mName=K, sName=Singh]
   Address:
   Address [streetName=Alipore, pincode=700027, state=West Bengal,
   cityOrVillage=Kolkata, houseNameOrBuildingName=KrishnaKunj]
```

Homework

1. Student–Department Aggregation
Design a Java program to demonstrate **aggregation** where a Student class *has a* Department class.
- Department should store department code, department name, and HOD name.
- Student should store roll number, name, and a reference to Department.
- Provide constructors, getters, setters, and toString() methods.
- Allow input of details for multiple students using an array.
- Display all student details in a formatted manner.
  Explain clearly why this relationship is aggregation and not inheritance.

2. Employee–Address Aggregation with Menu-Driven Program
Create a Java application where an Employee class aggregates an Address class.
- Employee contains employee ID, name, salary, and address.
- Address contains house number, street, city, state, and pincode.
- Implement a **menu-driven program** with options to:
  1. Add employee details
  2. Display all employees
  3. Search employee by ID
- Use proper constructors and override toString().
  Demonstrate how the Address object can exist independently of Employee.

3. School–Student–Name Aggregation
Write a Java program showing **multiple levels of aggregation**.
- Name class stores first, middle, and last name.
- Student class aggregates Name and stores roll number and class.
- School class aggregates multiple Student objects.
- Accept input for at least three students.
- Display school-wise student details.
  Explain object life cycle independence in aggregation.

4. Book–Author Aggregation
Design a program where a Book class aggregates an Author class.
- Author contains author name, email, and nationality.
- Book contains ISBN, title, price, and author details.
- Create multiple Book objects sharing the same Author object.
- Print all details clearly.
  Explain how aggregation helps in **code reuse** in this scenario.

5. University–Professor Aggregation Using Arrays
Develop a Java program where a University class aggregates a list of Professor objects.
- Professor class stores ID, name, subject, and experience.
- University stores university name and an array of professors.
- Input details dynamically using Scanner.
- Display professor details university-wise.
  Justify why inheritance is not suitable here.

6. Customer–BankAccount Aggregation
Write a Java program demonstrating aggregation between Customer and BankAccount.
- A customer may have multiple bank accounts.

- BankAccount stores account number, account type, and balance.
- Customer stores customer ID, name, and an array of accounts.
- Provide methods to deposit, withdraw, and display balances.
  Explain aggregation vs composition in this context.

7. Library–Book Aggregation

Create a Java program for a Library that aggregates Book objects.

- Book contains book ID, title, author, and price.
- Library contains library name and a collection of books.
- Implement methods to add books, display all books, and search by title.
  Explain how books can exist even if the library object is destroyed.

8. Company–Project Aggregation

Write a Java application where a Company aggregates multiple Project objects.

- Project stores project ID, project name, and duration.
- Company stores company name and list of projects.
- Input project details and display them company-wise.
  Explain how aggregation reduces tight coupling between classes.

9. Hospital–Doctor Aggregation

Design a program demonstrating aggregation between Hospital and Doctor.

- Doctor contains doctor ID, name, specialization, and consultation fee.
- Hospital contains hospital name and list of doctors.
- Allow adding and displaying doctor details.
  Explain how aggregation supports flexibility and maintainability.

10. Student–Course Aggregation with Command-Line Arguments

Create a Java program where Student aggregates Course.

- Course contains course code, course name, and duration.
- Student contains roll number, name, and course details.
- Number of students should be passed using **command-line arguments**.
- Accept and display all details properly.
  Explain why this relationship is **HAS-A** and not **IS-A**.